FinalSay - Multi-Institution Notice Verification and Provenance Platform

1. Abstract

Students often receive examination and administrative notices as forwarded PDFs, images or copied messages. These may be incomplete, altered or outdated, while institutional portals vary in structure and document quality. FinalSay is a locally runnable notice-verification prototype that collects public notices from three configured university sources and compares a student submission with current notices for the selected institution. A React and TypeScript progressive web app connects to a FastAPI and SQLAlchemy backend. Source-specific adapters fetch allowlisted portals; PDF text extraction and bounded OCR feed normalization, selected-field extraction and personal-data redaction. MiniLM embeddings retrieve likely candidates, and a DeBERTa natural-language-inference model compares claims. Seven relationship categories, evidence gates and reviewer workflows are implemented. SHA-256 fingerprints, Merkle proofs and a local append-only hash chain provide tamper evidence; a Solidity anchoring contract is tested locally but has not been publicly deployed.

The verified SQLite snapshot contains 2,284 official notice rows (2,164 current and 120 historical) and two student-submission records. A modified University of Delhi test submission retrieved a same-topic listing but was conservatively returned as UNRESOLVED because the dates conflicted and the official document text was unavailable. This is not a verified SUPERSEDED result. The current backend suite has 233 passing tests, the frontend production build passes, four Foundry tests pass, and SQLite integrity is OK. Live source polling remains degraded; public-chain deployment and real-user validation are not established.

2. Introduction and Objectives

The task is to help a student check whether a forwarded notice matches a current publication from the student's institution and whether its important details have changed. Manual checking requires portal search, document retrieval and comparison of dates, scope and actions. A later notice can revise an earlier one, add information without replacing it, or refer to a different event. Similarity alone cannot decide among these cases.

FinalSay aims to collect notices from a bounded set of official portals, preserve source identity and versions, extract usable text, retrieve institution-scoped candidates, compare claims, expose supporting source evidence and route uncertain cases for review. The current scope is a local prototype with three enabled sources: Visvesvaraya Technological University (VTU), University of Delhi Examination and University of Mumbai. It is not a general web crawler or a substitute for institutional confirmation. No production hosting, verified real-user accuracy, or public Polygon transaction is established.

Project objectives

- Ingest official portal listings and documents with source URLs, retrieval logs and extraction-quality labels.
- Accept student text, PDF or image submissions and normalize, redact and fingerprint extracted content.
- Retrieve current notices for the student's chosen institution and compare them with semantic, lexical and temporal evidence.
- Provide protected student, issuer, reviewer and administrator workflows, plus locally verifiable notice provenance.

3. System Architecture and Technology Stack

The PWA sends authenticated requests to FastAPI. Role checks and institution scope are enforced at the API. The service coordinates source polling, notice extraction, candidate retrieval, comparison and review. SQLAlchemy persists users, institutions, notices, versions, relation edges, review cases and provenance data. The default local database is SQLite; PostgreSQL is supported by configuration. Optional Polygon anchoring is isolated from the local provenance flow.

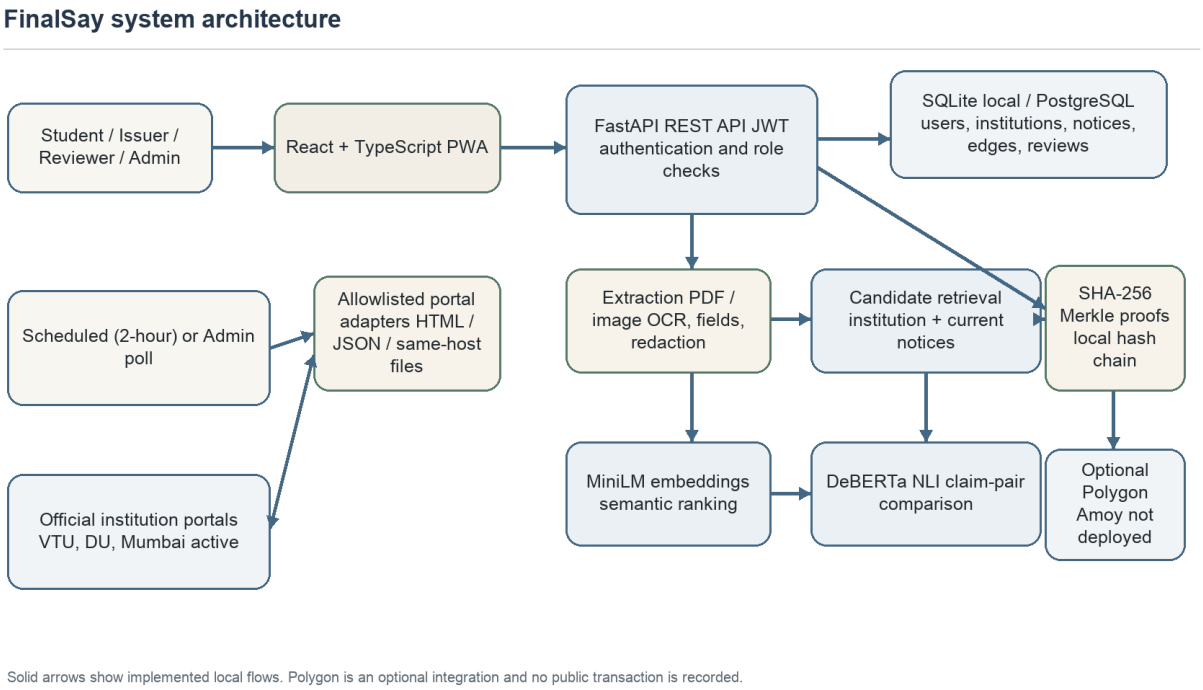


Figure 1. Implemented local architecture. Polygon Amoy is optional; no public deployment or transaction is recorded.

Technology stack

Layer	Technology	Role in FinalSay
Web client	React 18, TypeScript, Vite, PWA	Student and role-specific workflows; API client and browser install support.
API and access	Python 3.11, FastAPI, JWT, Pydantic	REST endpoints, validation and role/institution authorization.
Persistence	SQLAlchemy 2, SQLite; PostgreSQL option	Notice records, source identity, versions, users, comparison edges and reviews.
Document processing	pypdf, PyMuPDF, Tesseract OCR	Extract PDF text first; rasterize/OCR when a usable text layer is missing.
Models	all-MiniLM-L6-v2; cross-encoder/nli-deberta-v3-small	Semantic candidate ranking and NLI claim comparison.
Integrity and chain	SHA-256, Merkle proofs, local hash chain; Solidity / Foundry	Local tamper evidence and tested optional root-anchoring contract.

Portal adapters are source-specific and allowlisted. The scheduled poll is configured for every two hours, and administrators can request manual polling. The configured cadence is not evidence that every scheduled fetch succeeds.

4. Methodology and Implementation

4.1 Notice collection, identity and versioning

Three enabled adapters read the VTU examination catalog, University of Delhi Examination notifications and University of Mumbai circular portal [15]. Adapters support portal-specific HTML or JSON parsing, bounded pagination/backfill, host checks, response-size/time limits, rate spacing, retries and per-source run logs. Document links are retrieved only when allowed by the configured source rules. The current database has records for six institution profiles, of which three are enabled.

A stable portal identifier or normalized source URL acts as a source key. Repeated polling of an unchanged source document updates retrieval metadata and creates no duplicate. When the same source key has changed bytes, the older notice is retained as historical and a linked current version is inserted. A newly posted notice at a different URL is stored as a separate notice; discovery of its relationship to an older URL depends on candidate retrieval and is not guaranteed by versioning. The SQLAlchemy schema stores institutions, users and issuer memberships; versioned Notice rows; extracted fields and document-extraction sidecars; submission-to-candidate RelationEdge records; ReviewCase records; and scrape progress, run logs and Merkle data.

4.2 Document text, extraction and redaction

The pipeline extracts embedded PDF text with pypdf before invoking OCR; PyMuPDF rasterization and Tesseract OCR are bounded by file, page and time limits [12-14]. Extracted text is normalized; selected identifiers and contact details are redacted; issuer, notice date, deadline, audience and action fields are stored with extraction metadata. Publication dates from portal listings are tracked separately from dates found inside a notice. Quality flags distinguish extracted text, listing-only records, unextractable files and older or unverified records. These flags describe the extraction path and do not certify OCR correctness.

4.3 Candidate retrieval and NLI comparison

Candidate retrieval is scoped to current official notices for the student's institution. Sentence embeddings from all-MiniLM-L6-v2 rank semantic relevance [1,2]; lexical overlap and informative title similarity supplement the score, which is useful when a portal listing has little body text. The cross-encoder/nli-deberta-v3-small model evaluates premise/hypothesis pairs for entailment, contradiction and neutrality [3-5]. NLI scores are model outputs, not calibrated probabilities or direct measures of official authenticity.

The four primary user-facing outcomes are consistent, contradictory, superseded and unresolved; corrected, extended and cancelled are additional relationships. Same-topic update cues prioritize likely newer notices. Date-category conflicts and listing-only evidence can gate a result to unresolved and create a reviewer case. Scores are comparative outputs, not calibrated probabilities [3-5].

Relationship	Implemented interpretation
Consistent	Relevant claims align and the comparison finds no supported material conflict.
Contradictory	Relevant claims conflict without sufficient evidence of a formal replacement.
Superseded	A newer official revision/reschedule signal and conflicting event detail indicate replacement.
Corrected	A correction or erratum cue identifies a corrected notice detail.
Extended	An official extension cue changes a deadline or applicable period.
Cancelled	Official cancellation language withdraws an event or requirement.
Unresolved	Evidence is weak, ambiguous, unavailable or gated for human review.

Implementation progressed through source and requirements analysis, schema/API design, adapter and document-processing work, retrieval and comparison integration, role and provenance workflows, and local verification/deployment preparation. The repository contains no reliable dated milestone log, so completion dates are not assigned.

4.4 Provenance, contract and application integration

FinalSay computes a SHA-256 fingerprint for notice content and selected fields using the SHA-256 standard [6]. Daily Merkle roots and per-notice inclusion proofs support local verification; an append-only local hash chain records root history. These mechanisms can show consistency with stored hashes, but a database administrator who controls all local state can replace both content and proofs. They are local tamper evidence, not independent public timestamps.

The Solidity contract stores notice-batch roots and associated metadata with publisher authorization and duplicate/empty-value checks [10,11]. The Web3 backend path, local/mock behavior and Foundry tests are implemented. The current database contains local roots only. No Polygon Amoy contract address, deployment receipt or on-chain transaction is evidenced; testnet deployment remains a future credentialed step.

The React PWA supports student registration and college selection, notice browsing and filtering, text/PDF/image submission, result evidence, alerts, and protected issuer/reviewer/admin routes [7-9]. The client communicates with REST endpoints using JSON or multipart uploads; form-based login returns a JWT sent in the Authorization header. Students are scoped to their saved institution and own submissions; issuer publishing requires an active institution membership; reviewer annotations are account-scoped; administrators manage source profiles and polling. The service persists the comparison and returns the candidate, source and review status to the UI.

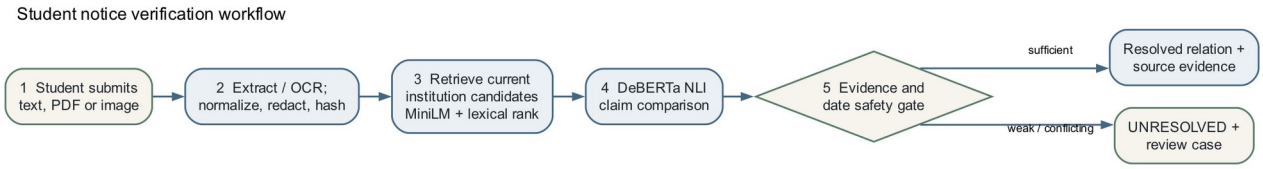


Figure 2. Student-submission workflow from ingestion through evidence gating and review.

Blockchain provenance path

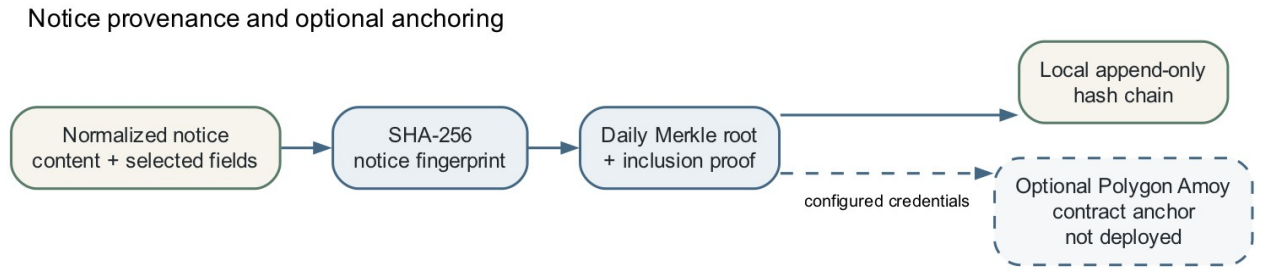


Figure 3. Local SHA-256, Merkle and hash-chain path; Polygon Amoy anchoring is optional and not deployed.

Implementation boundary

The system can discover an update published at a different URL by matching the submitted notice against current institution-scoped candidates. It does not assert that every separate portal entry is an update, and it cannot promise correct classification when the official file cannot be retrieved or extracted. The comparison result must be read together with the source link, extraction quality and reviewer state.

5. Results and Testing

The following counts come from a read-only query of backend/finalsay.db. SQLite PRAGMA integrity_check returned “ok”. Counts describe the local database snapshot, not the current number of notices on the institutions’ websites.

Institution source	Current	Historical	Status
VTU	720	46	Active and polling enabled
University of Delhi Examination	415	7	Active and polling enabled
University of Mumbai	990	33	Active and polling enabled
Thapar Institute of Engineering and Technology	24	26	Disabled
Indian Institute of Technology Bombay	15	8	Disabled
NIT Jalandhar	0	0	Disabled profile
Total official notices	2,164	120	2,284 official rows

The database also contains two student-submission rows. Their presence does not establish a real-user validation study. Six institution profiles are configured; only the three named sources above are enabled.

Extraction quality for active-source current notices

Quality flag	Count	Share	Interpretation
Extracted	1,338	62.96%	Document text was extracted; accuracy is not guaranteed.
Listing-only	269	12.66%	Portal catalog text is available; underlying document text is absent.
Unextractable	1	0.05%	A source document could not be extracted.
Unknown	517	24.33%	Quality was not reliably established.

The denominator is 2,125 current notices from the three enabled institutions. “Unknown” is not evidence that a PDF was successfully read. All current selected-source rows have a stored publication-date value, but one VTU raw date remains unnormalized. The quality mix limits how confidently the comparison service can interpret all candidate documents.

Polling and integrity evidence

Evidence	Observed result
Recorded source runs	301 total: 49 ok, 145 partial, 107 error.
Latest active-source records	VTU and Delhi show DNS-resolution errors; Mumbai shows a socket Bad file descriptor error.
Idempotency/versioning	Covered by automated tests for unchanged polling and changed same-key documents; current database audit found no duplicate current source keys.
Local provenance	105 local roots and 105 local anchor-chain blocks; 54,516 proof rows in the snapshot.
Public chain	No Polygon root, contract address, transaction hash or receipt found.

5.1 Testing and student comparison evidence

Area	Verified result	Limit
Backend	233 tests passed in the current backend suite.	Most external portal paths use fixtures/mocks; passing tests do not establish live availability.
Frontend	TypeScript production build passed; Vite/PWA assets generated.	Build success is not user-acceptance testing.
Smart contract	Foundry: 4 passed, 0 failed, 0 skipped; local Solidity compile passed.	No funded transaction or public deployment.
Database	SQLite integrity check: ok.	Structural integrity does not validate source authenticity or ML correctness.

Edited University of Delhi notice replay

A modified student notice based on a University of Delhi exam-form notification was replayed against current official candidates. Retrieval selected a later same-topic listing (notice #381); the NLI component alone favored CONSISTENT. The comparison service detected conflicting deadline dates and that the candidate was listing-only, then returned UNRESOLVED for review. The listing's official PDF text was not available for evidence. This is evidence that the integration safety gate is active; it is not a successful SUPERSEDED classification and does not validate the model on real student traffic.

The database contains one open reviewer case and 45 benchmark pairs, but zero completed reviewer annotations at the inspected snapshot. Cohen's Kappa cannot be calculated without independent overlapping annotations. The stored submissions likewise do not establish a real-user validation study.

Result interpretation

FinalSay's principal implementation outcome is a connected local pipeline: official-source records can be persisted and versioned; student input can be processed; current notices can be retrieved; comparison evidence, uncertainty and review state can be returned; and local integrity proofs can be checked. The current operational evidence is weaker than the implementation coverage: the latest source runs are failing, a substantial share of active notices has unknown or listing-only text, and the demonstrated revision case remains unresolved.

6. Challenges and Limitations

Challenge	Implemented mitigation	Remaining limitation
Portal access and layout changes	Host allowlists, bounded fetches, retries, source-specific adapters, run logs and two-hour schedule.	Latest active-source polls fail with DNS/socket errors; historical backfills and freshness are incomplete.
PDF/OCR and dates	Text-first extraction, bounded OCR, quality flags, separate publication-date parsing.	Scanned tables, missing files, OCR errors and 517 unknown-quality current records remain.
Semantic and temporal ambiguity	Institution-scoped MiniLM retrieval, title/lexical signals, DeBERTa NLI, date-conflict and evidence gates.	NLI can prefer a misleading label; no correct SUPERSEDED outcome or representative human gold set is established.
Blockchain deployment	Local hash chain/Merkle proofs and tested Solidity/Web3 integration.	No public Amoy deployment; RPC/provider and secure wallet configuration are still required.
Operational readiness	Local build, automated tests and PostgreSQL/hosting configuration paths exist.	No production hosting, monitoring/alerting, backup-restore evidence, or real-user validation.

The prototype has no measured production throughput, calibrated accuracy, confidence intervals or user-feedback results. Scraper success at one portal does not generalize to another. A local digest proves only that content matches a recorded digest; it does not prove that a notice is authentic or currently endorsed by the institution. Source evidence and uncertainty must remain visible to students.

7. Conclusion and Future Scope

FinalSay implements a multi-institution workflow for collecting official notices, extracting and redacting document text, retrieving likely candidates, comparing notice claims and retaining local provenance. The current implementation spans the student interface, role-protected APIs, portal adapters, database versioning, semantic retrieval, NLI comparison, reviewer workflows and a locally tested Solidity contract. The verified database contains 2,284 official rows; automated backend, frontend and contract checks pass.

Before a public pilot, the project should restore reliable polling and complete source backfills; retrieve and extract documents behind listing-only candidates; validate date-aware revisions on real, institution-sourced notice pairs; and obtain independent human annotations with Cohen's Kappa from actual labels. It should then add operational monitoring, backups, HTTPS hosting and user testing. Polygon Amoy anchoring should be attempted only after secure backend wallet configuration and a confirmed deployment transaction. Current evidence supports a substantial local prototype, not a production-validated service.

8. References

[1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” EMNLP-IJCNLP, 2019. https://aclanthology.org/D19-1410/

[2] Sentence Transformers, “all-MiniLM-L6-v2 model card.” https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed 12 Nov. 2026).

[3] A. Williams, N. Nangia and S. R. Bowman, “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” NAACL-HLT, 2018. https://aclanthology.org/N18-1101/

[4] P. He, X. Liu, J. Gao and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” arXiv:2006.03654, 2020. https://arxiv.org/abs/2006.03654

[5] Sentence Transformers, “cross-encoder/nli-deberta-v3-small model card.” https://huggingface.co/cross-encoder/nli-deberta-v3-small (accessed 12 Nov. 2026).

[6] National Institute of Standards and Technology, FIPS PUB 180-4, Secure Hash Standard, 2015. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[7] FastAPI, “Tutorial - User Guide.” https://fastapi.tiangolo.com/tutorial/ (accessed 12 Nov. 2026).

[8] SQLAlchemy, “SQLAlchemy 2.0 ORM Documentation.” https://docs.sqlalchemy.org/en/20/orm/ (accessed 12 Nov. 2026).

[9] React, “Learn React.” https://react.dev/learn (accessed 12 Nov. 2026).

[10] Solidity, “Solidity 0.8.24 Documentation.” https://docs.soliditylang.org/en/v0.8.24/ (accessed 12 Nov. 2026).

[11] Foundry, “Forge Documentation.” https://getfoundry.sh/forge/ (accessed 12 Nov. 2026).

[12] pypdf, “Extract Text from a PDF.” https://pypdf.readthedocs.io/en/stable/user/extract-text.html (accessed 12 Nov. 2026).

[13] Tesseract OCR, “Tesseract User Manual.” https://tesseract-ocr.github.io/tessdoc/ (accessed 12 Nov. 2026).

[14] PyMuPDF, “PyMuPDF Documentation.” https://pymupdf.readthedocs.io/en/latest/ (accessed 12 Nov. 2026).

[15] Official institutional notice portals: VTU Examination, https://vtu.ac.in/category/examination/; University of Delhi Examination, https://exam.du.ac.in/exam/index.php/notifications; University of Mumbai, https://mu.ac.in/circular.php (accessed 12 Nov. 2026).